A 16-variable Diophantine encoding of ZFC consistency

Research contribution prepared for Vladimir Reshetnikov
and the ProveIt repository

October 22, 2026

Abstract

We specify a fixed polynomial $P \in \mathbb{Z}[\mathbf{x}]$ in sixteen positive integer unknowns such that ZFC proves

$$\left(\nexists \mathbf{x} \in \mathbb{Z}_{>0}^{16} : P(\mathbf{x}) = 0\right) \quad \longleftrightarrow \quad \mathrm{Con}(\mathsf{ZFC}).$$

The polynomial has exact degree 156 and a complete straight-line evaluation using $73 = 41\mathrm{M}{+}32\mathrm{A}$ arithmetic operations. An equivalent system has seven equations, maximum degree 60, and a $65 = 35\mathrm{M} + 30\mathrm{A}$ common evaluation circuit. We also give an equation in 44 unrestricted integer unknowns and an explicitly listed quadratic system in 83 integer unknowns; summing the latter's 46 squared residuals gives a single quartic equation.

The construction specializes the repository's native universal Diophantine representation to one fixed input. Its principal new step replaces the variable-input Pell decoder with the affine factor $3W + 1 - 3W_0$: an integer unit congruent to 1 modulo 3 must be 1. The fixed coefficients are completely specified by finite formulas and standalone code starting from the literal K215 table. Their enormous expanded numerals are not materialized. The proof composes the inherited K215 calibration and native compiler theorem with new normalization, input-elimination, and algebraic arguments. The new composed theorem is a mathematical proof, not a newly kernel-checked Lean or Rocq endpoint. No global minimality claim is made.

Contents

1 Result, scope, and the meaning of simplicity **3**

2 The polynomial itself **4**

3 From the literal K215 table to a fixed input **5**
3.1 A smaller normalization by allowing both blank margins 5

4 Closed definitions of the coefficients **6**
4.1 The ordered local window list 6
4.2 A finite arithmetic recipe 7
4.3 What is explicit, and what was actually evaluated 8

5 Correctness of the fixed-input reduction **8**
5.1 An affine input guard 9
5.2 Outer bounds before decoding anything 9
5.3 Restoring the auxiliary quotient coordinates 9
5.4 The main Pell rank and the remaining signs 10
5.5 Recovering a power without using the omitted input block 10
5.6 Restoring the three deleted input witnesses 11

6 Operation counts and exact degrees **11**
6.1 The complete evaluator 11
6.2 Uniform exact degree 12

7 Unrestricted integers and a small-degree system **12**
7.1 One equation in forty-four unrestricted integers 12
7.2 Forty-six quadratic equations in eighty-three integers 13

8 Why the equivalence is a theorem of ZFC **13**

9 Verification, reproducibility, and remaining work **14**

A The explicit quadratic equations **15**

1 Result, scope, and the meaning of simplicity

Fix the ordinary syntactic consistency sentence

$$\mathrm{Con}(\mathsf{ZFC}) \equiv \neg\exists n \in \mathbb{N}\ \mathrm{Proof}_{\mathsf{ZFC}}(n, \bot),$$

for the finite-derivation coding used by the K215 calibration. Changing to a standard effectively intertranslatable coding does not change the result: the translations and their correctness can be proved in ZFC. We encode a refutation of ZFC, not a proof of its consistency and not a semantic truth predicate.

The source snapshot used throughout is `cdd582bec13f908df3711fa9e05d1ddac368a5db`. Its K215 endpoint says that the exact 215-state binary machine halts on a blank tape if and only if ZFC has a finite refutation [1, 2]. The repository also contains a complete native Diophantine compiler and several carefully distinguished operation/degree/witness tradeoffs [3, 4, 5, 6]. Those are substantial inherited results; the present work does not rebrand them as new discoveries.

Theorem 1.1 (Main construction)**.** *Let the fixed integers in Section 4 be constructed from the literal K215 table by the prescribed finite recipe. Let P be the polynomial in Section 2. Then*

$$\mathsf{ZFC} \vdash \left[\exists \mathbf{x} \in \mathbb{Z}_{>0}^{16}\ P(\mathbf{x}) = 0\right] \longleftrightarrow \neg\,\mathrm{Con}(\mathsf{ZFC}).$$

It has exact degree 156*, and the supplied complete evaluator has* 41 *multiplications and* 32 *additions or subtractions.*

The following alternatives are included in the archive. All operation counts charge a binary addition, subtraction, or multiplication as one operation, including multiplication by a fixed numeral. Fixed numerals and reuse of earlier results are free in this arithmetic-circuit model. The bit lengths and the cost of constructing the coefficients are not included in these counts.

Representation	Unknowns	Equations	Degree	Arithmetic
Scaled product P	16 positive	1	156	73
Unscaled product U	16 positive	1	144	74
Comparison system	16 positive	7	60	65
Direct sum of squares	16 positive	1	120	85
Integer substitution	44 integers	1	312	171
Quadratic system	83 integers	46	2	not minimized
Sum of quadratic squares	83 integers	1	4	not minimized

"Simplest" has no invariant meaning without an objective. Number of unknowns, degree, circuit length, coefficient height, and proof complexity give different orderings. Nine-natural-variable universal constructions exist, with extremely large degrees; this is not a variable-count record [9, 10]. Our main choice optimizes a short evaluator and a readable proof within the supplied repository framework. The seven-equation version is preferable when a system is acceptable and the number of arithmetic operations matters more than having one equation. The quadratic version provides a small-degree system. A concrete unsatisfiable linear integer system has an effective finite arithmetic proof of unsatisfiability, so such a system cannot furnish an independent nonexistence statement for consistent ZFC. We make no global degree, variable-count, or equation-count minimality claim.

The old 84-operation construction carries a varying ordinary input. The present coefficients encode one fixed computation, including its fixed input. The two operation counts therefore describe different interfaces. The improvement is a specialization, not an assertion that the old varying-input universal frontier has been replaced.

2 The polynomial itself

The sixteen unknowns, all strictly positive for the principal theorem, are

$$J, F, \alpha, t, f, h, i, T, s, w, \tau, \eta, \zeta, y, Z, \gamma.$$

Use the six fixed integer coefficient ports

$$\beta = B - 1, \qquad K_0, \qquad e = 2d, \qquad M_C, \qquad \widehat{M}_F = M_F + B - 1, \qquad c_0 = 1 - 3W_0.$$

Here $B = 2^d$, $W_0 = 2^{b+2d}$, and M_F is the *native* verification mask. The port $\widehat{M}_F$ is its shifted value. These are fixed coefficients, not existential variables. In particular c_0 is negative; this does not conflict with positivity of the unknowns.

Define the following polynomial abbreviations:

$$q = \beta J + 1, \qquad X = wq, \qquad Y = sq^3, \qquad E = XY, \tag{1}$$

$$k = \eta + \zeta, \qquad c = kY + \eta, \qquad a = E + Y, \qquad H = 4a + 3, \tag{2}$$

$$\Delta = a^2 + H, \qquad D = X + ac + \gamma H, \tag{3}$$

$$C = q - F - Z - \alpha - e, \qquad W = C - Z, \tag{4}$$

$$R = (q^2 - Z - qF)(q^2 - 1) + (M_C + q\widehat{M}_F)J, \tag{5}$$

$$L = E(kY), \qquad S = \Delta i^2 c^4, \qquad A_0 = \Delta^2 i^2 c^4, \tag{6}$$

$$V = c(Tf - 1) - Rf^2. \tag{7}$$

Equations (1)–(7) are definitions, not additional constraints or additional unknowns. In particular R, C, W are computed polynomials; they are not assumed positive off the zero set.

The seven factors are

$$N_1 = \tau^2 - L(L + k), \qquad N_2 = D^2 - \Delta c^2, \tag{8}$$

$$N_0 = 3W + c_0, \qquad N_3 = A_0(V^2 - y^2) + y^2, \tag{9}$$

$$N_4 = k - hE - R, \qquad N_5 = (K_0 + w)C + q - F - t(q - 1), \tag{10}$$

$$N_6 = f^2 - S. \tag{11}$$

Our two product polynomials are

$$\boxed{U = N_1 N_2 N_0 N_3 N_4 N_5 N_6 - 1, \qquad P = \Delta U.} \tag{12}$$

The required equation is $P = 0$ on $\mathbb{Z}_{>0}^{16}$. Everything in (12) is an ordinary polynomial with integer coefficients. There is no exponent whose value depends on an unknown. The exponents occurring in the coefficient recipe, such as $b + 2d$, are fixed integers.

The equivalent seven-equation system is especially easy to state:

$$\boxed{W = W_0, \qquad N_1 = N_2 = N_3 = N_4 = N_5 = N_6 = 1.} \tag{13}$$

The nontrivial assertion is that a product zero really forces these seven equalities; multiplying factors is not automatically a valid replacement for a system. Section 5 proves this assertion.

3 From the literal K215 table to a fixed input

The canonical dense K215 file has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

Its working states are $0, \ldots, 214$, its separate halt is 215, and its alphabet is $\{0, 1\}$. The exact file is included, together with the generator that verifies this hash before using it.

The native tableau compiler expects a first step that writes a flag and stays in a different nonhalting state. Add four initialization states $0, 1, 2, 3$; represent the old state q by $q + 4$, and use a third tape symbol 2. The halt becomes 219. The six exceptional initialization transitions are:

State	Read	Next	Write	Move
0	1	1	2	stay
1	2	2	2	left
2	1	2	0	left
2	0	3	0	right
3	0	3	0	right
3	2	4	0	stay

Every old transition is copied under $q \mapsto q + 4$. All other entries are identity stay transitions, including the absorbing halt entries. This defines all 660 rows of a 220-state, three-symbol table. There are 219 working states; this normalization is not a smaller binary Busy Beaver candidate.

Lemma 3.1 (Blank initialization)**.** *For every $u \geq 1$, on a tape containing ones exactly at $-u, \ldots, 0$, the normalized machine reaches the blank tape, head position 0, and state 4 after exactly $2u + 4$ transitions. Its subsequent execution is the literal K215 blank execution.*

Proof. The first two steps mark the origin and move left. State 2 erases the u left-hand ones, then moves right from the next blank. State 3 crosses the erased cells, erases the origin flag, and enters state 4. Counting these steps gives $2u + 4$. The remaining claim is induction on the copied transition table. □

At ordinary compiler input $x = 1$, the marked displacement is $2x = 2$, the initial phase word has three headless input ones, and the head cell contributes one more one. Thus $u = 3$, and initialization takes ten transitions. This normalized machine accepts every admissible positive input if K215 halts, and no such input otherwise. No unknown halting time occurs in its finite table.

3.1 A smaller normalization by allowing both blank margins

A folded one-sided simulation is unnecessary. The actual local tableau rules already permit an initial row

$$L^{a_\ell} I^u Q R^{a_r}$$

with arbitrary positive blank margins. The inherited completeness presentation fixed $a_r = 2$ after choosing a one-sided machine; the local rules contain no such length restriction.

Given any finite accepting run of the two-sided normalized machine, let ℓ, r be its extreme head coordinates, including initialization. Choose

$$a_\ell \geq \max(2, 1 - u - \ell), \qquad a_r \geq \max(2, r + 1).$$

The width $h_* = a_\ell + u + a_r + 2$ can then be any sufficiently large power of five, by enlarging the left margin. Choose independently a sufficiently large power-of-five time height $H_* \geq T_* + 2$ and pad with the absorbing halt. This retains all width, height, and dummy-bit padding required by the inherited compiler.

Place the distinguished Start at the last I, in column $x_S = a_\ell + u$, and End at the first I. Their separation is $u - 1 = 2$. The usual helical extension

$$\mathcal{T}(z, t) = G[z \bmod h_*,\ (t + \lfloor z/h_* \rfloor) \bmod H_*]$$

has periods $(h_*, -1)$ and $(0, H_*)$. Its cyclic index $-(z - x_S) - h_*(t - 1)$ modulo $h_* H_*$ puts Start at 0 and End at 2. The vertical boundary carries no payload, so the seam preserves the same local rules. Conversely those rules recover a finite strip, a unique initialized head, a genuine run without boundary escape, and a halted head before the next horizontal boundary. This soundness argument never needed a one-sided tape.

This is a new elementary extension of the existing completeness argument. Its proof and a two-sided accepting toy tableau check are included separately; it is not attributed to an existing formal kernel theorem.

4 Closed definitions of the coefficients

The coefficient recipe is fully effective and finite. Its input is the complete normalized table just defined, not an unspecified universal-machine index. The supplementary source `src/k215_constants.py` fixes all ordering conventions and implements the formulas below. A reader can therefore recover exactly one polynomial, rather than an existentially chosen polynomial.

4.1 The ordered local window list

The tile alphabet has vertical and horizontal boundary symbols; an unphased interior symbol for each tape symbol paired with either no head or one of the 220 states; and four initialization-phase symbols L, I, Q, R. In this order it has

$$a_* = 2 + 3(220 + 1) + 4 = 669$$

members. Enumerate allowed 3×3 windows with the prescribed Start window first, End second, then all other accepted tuples in lexicographic order. The complete predicate is the literal `local_valid` function. It enforces propagated boundaries, the phase graph $L \to L, I$, $I \to I, Q$, $Q \to R$, $R \to R$, the exact machine update, no boundary crossing, and a halted head just below the next horizontal boundary. Neighbor overlap enforces the same conditions at every window center.

With the supplied tile order the two distinguished tuples are

$$\text{Start} = (1, 1, 1, 666, 666, 667, 223, 223, 446),$$
$$\text{End} = (1, 1, 1, 665, 666, 666, 2, 223, 223).$$

Let $\mathcal{W} = (w_0, \ldots, w_{k_*-1})$ be this ordered finite list. An intentionally simple enumeration inspects

$$669^9 = 26843247761153995367677629$$

candidate tuples. It is not a feasible routine to run for this machine. It nevertheless defines a particular finite list and contains no unbounded search for a computation or a refutation.

The four corner entries do not enter the local predicate. Counting the five-cell crosses instead gives an exact, quickly evaluable cardinality without constructing the window list:

$$k_* = (896017122 + 235501665872 + 11505)669^4 = 47353022820782362885779.$$

The three summands count boundary, unphased, and phased crosses. The supplied counting function derives them from the actual transition table; its derivation and an independently enumerated small fixture are included in the coefficient note.

4.2 A finite arithmetic recipe

Here all exponents and summation indices are fixed data. Write $A_* = 2^{\max(2,\mathrm{bitlength}(k_*+1))}$. For $r, c \in \{0,1,2\}$ and $0 \le v < a_*$, put

$$p(r,c,v) = k_* + 3a_* + (2-r)3a_* + (2-c)a_* + v.$$

Initialize selector coefficients $c_j = 1$ for $j < k_*$ and $\mu = A_* - 2$. Enumerate the $9a_*$ triples (r,c,v) lexicographically, with index $j = 1, \ldots, 9a_*$. Set $c_{p(r,c,v)} = A_*^j$, add A_*^j to μ, and add it to each selector coefficient whose window has value v at position (r,c). Reserve four more powers $A_*^{9a_*+1}, \ldots, A_*^{9a_*+4}$ as anchor coefficients, adding each to μ and to every selector coefficient.

Set $m = 2\,\mathrm{popcount}(\mu) + 12a_* + 2$. Until $m + 1 \ge k_* + 9a_* + 5$, add the next unused power of A_* to μ and increase m by two. Add $m + 1 - k_* - 9a_* - 4$ consecutive dummy positions starting at $k_* + 12a_*$. Let $\mathcal{E}_0$ contain the selectors, copy positions, and dummies. Set $M_* = \max \mathcal{E}_0 + 3a_* + 1$; add anchors $M_*, 3M_*, 9M_*, 27M_*$, with the four reserved coefficients, to obtain $\mathcal{E}$. Undefined coefficients are zero. If $E_* = \max \mathcal{E}$, define

$$H_c = E_* + 24M_* + 3a_* + 1, \qquad T_1 = H_c + 2E_* + a_* + 1,$$
$$T_2 = T_1 + 2E_* + 1, \qquad g = T_2 + E_* + 1.$$

Let L_* be the least power of five at least $g + E_* + 1$. In a formal radix z, form

$$D_C(z) = z^{3a_*} + z^{H_c + a_*} + z^{8M_*} + z^{24M_*} + \sum_e c_e (z^{T_1 - e} + z^{T_2 - e}),$$

$$D_R(z) = z^{H_c},$$

$$M_{F,0}(z) = \mu(z^{T_1} + z^{T_2}) + \sum_{r<2, c<3, v<a_*} z^{p(r,c,v)} + \sum_{r<3, c<2, v<a_*} z^{H_c + p(r,c,v)} + z^{9M_*} + z^{27M_*}.$$

Add z^g to D_C precisely when $2D_C(2) - 2^{H_c} \equiv 0 \pmod 5$. After this correction the difference $2D_C(2) - 2^{H_c}$ is nonzero modulo five. Put

$$\mathcal{M} = (|\mathcal{E}| + 2)\left(2\sum_e c_e + 6\right).$$

Choose b to be the least power of five satisfying $2^b \ge \max(4\mathcal{M} + 8, 2\mu + 4, 16)$, and set $z_* = 2^b$, $d = bL_*$, $B = 2^d$. If e_* is the first dummy position, the final constants are

$$M_C = B - 1 - \sum_{e \in \mathcal{E},\ e \ne 1} z_*^e - 2z_*^{e_*}, \tag{14}$$

$$M_F = M_{F,0}(z_*) + 4, \qquad \widehat{M}_F = M_F + B - 1, \tag{15}$$

$$K_0 = D_C(z_*) + BD_R(z_*), \qquad W_0 = 2^{b+2d}, \tag{16}$$

$$\beta = B - 1, \qquad e = 2d, \qquad c_0 = 1 - 3W_0. \tag{17}$$

The implementation resolves all finite sums in exactly this order. The inherited compiler proves, among its other typing and synchronization facts,

$$0 < M_C, M_F < B - 1, \qquad \operatorname{popcount}(M_C) + \operatorname{popcount}(M_F) = d,$$
$$M_C \equiv 2 \pmod 4, \qquad M_F \equiv 4 \pmod 8.$$

The shift in (15) is essential.

4.3 What is explicit, and what was actually evaluated

The scalar parts of this recipe have been evaluated exactly:

Quantity	Exact value
Coefficient radix A_*	2^{76}
Zero-clause padding steps	23676511410391181435787
Number of dummy positions	1
Anchor unit M_*	47353022820782362895815
Cell width L_*	5^{36}
Radix-bit parameter b	5^{35}
Bit width $d = bL_*$	5^{71}

In particular the following large fixed coefficients are already available in short closed form:

$$\boxed{B = 2^{5^{71}}, \qquad W_0 = 2^{5^{35}+2\cdot 5^{71}}.} \tag{18}$$

For the determination of b, the target $2\mu + 4$ dominates the competing mass target: $\operatorname{bitlength}(\mu) = 17994148671897297895777713$, whereas the mass target has bit length at most 458130. The exact digit layout then makes 5^{35} the least admissible power of five. These statements use finite counting and integer arithmetic, with no window enumeration or machine-halting assumption.

The fixed coefficients are specified as exact finite integers by (14)–(17) and the completely specified window list. The archive contains their executable definition, the literal normalized table, its digest, exact scalar parameters, and the actual marker tuples. It does not contain expanded decimal or hexadecimal representations of the K215 coefficients. The default generator deliberately does not start the astronomical enumeration. Its opt-in generation modes execute the whole finite recipe in principle.

Thus this is a closed coefficient specification, rather than a materialized coefficient list. It is substantially more explicit than "take the index of a machine searching for a contradiction"; every table entry, finite predicate, ordering rule, and arithmetic formula is given. It is also a genuine limitation if one wants a small human-readable coefficient height. We do not claim such a height bound or optimality in that metric.

5 Correctness of the fixed-input reduction

We prove a reversible reduction to the exact normalized 85-operation parent of [5]. That parent has eighteen positive witnesses, with a variable ordinary input x. In its notation our γ is $\rho + \sigma$. Its variable-input block is a Pell norm with additional positive witness δ.

5.1 An affine input guard

Lemma 5.1. *If $U = 0$ on positive supplied coordinates, then $W = W_0$ and $N_1 = N_2 = N_3 = N_6 = 1$. Moreover $N_4, N_5 \in \{1, -1\}$ and $N_4 N_5 = 1$.*

Proof. Every factor of the integer product 1 is a unit. Since $N_0 = 3(W - W_0) + 1 \equiv 1 \pmod 3$, it cannot be -1; hence $N_0 = 1$ and $W = W_0$.

For the first norm put $v = XY^2 > 1$. Then $L(L + k) = v(v + 1)k^2$. The negative Pell equation $x^2 - v(v + 1)z^2 = -1$ has no positive solution when $v > 1$. Indeed, take a solution with least $z > 0$. Its inequalities $vz < x < (v+1/2)z$ show that multiplication by the inverse unit $(2v+1)-2\sqrt{v(v+1)}$ produces a solution with $0 < z' = (2v + 1)z - 2x < z$, a contradiction (take the absolute value of the new first coordinate if necessary). Thus $N_1 = 1$.

The integer Δ is 0 or 3 modulo four. Therefore neither $D^2 - \Delta c^2$ nor $f^2 - \Delta i^2 c^4$ can be -1 modulo four. Finally A_0 is a square, so N_3 is congruent either to y^2 or to V^2 modulo four and cannot be -1. The claimed unit signs follow. □

The affine guard saves one gate over the tempting positive guard $1 + (W - W_0)^2$. The constant $1 - 3W_0$ is evaluated once as a fixed coefficient. Omitting the multiplier three would be invalid: the second unit sign would then allow $W = W_0 - 2$.

5.2 Outer bounds before decoding anything

Because $W = W_0 > 0$ and $Z > 0$, equation (4) gives $C = Z + W_0 > 0$ and $F + Z < q$. Also $q \geq B \geq 16$. The actual native mask bounds, with the shifted source mask, imply

$$(2q - 1)(q^2 - 1) < R < q^4 - q^3, \qquad 3q + 1 < R - 2 < R + 2 < q^4. \tag{19}$$

For example $F + Z \leq q - 1$ implies $q^2 - Z - qF \geq q + (q - 1)Z \geq 2q - 1$. The upper bound follows from $F, Z \geq 1$ and the mask's bound below $2q^2 - 2q$. Consequently

$$E = wsq^4 \geq q^4 > R + 2, \qquad a = E + Y > R + 2. \tag{20}$$

These estimates establish $R > 0$ before any Pell-index or typed-word argument. Eliminating a positive packed-index variable without recovering such a bound would be a genuine soundness error.

5.3 Restoring the auxiliary quotient coordinates

From $N_6 = 1$ we have $f^2 = 1 + \Delta i^2 c^4$ and $f > c^2 > 2c$. As $R < a < \Delta$,

$$A_0 - (Rf^2 + c) = (\Delta - R)f^2 - \Delta - c \geq f^2 - \Delta - c > 0. \tag{21}$$

The auxiliary norm reads $A_0 V^2 - (A_0 - 1)y^2 = 1$. It excludes $V = 0$. If $v = |V| > 1$, then $v^2 - 1 = (A_0 - 1)(y^2 - v^2) > 0$, so $y \geq v + 1$ and $v \geq 2A_0 - 1$. On the other hand the literal formula for V and $T > 0$ give $V > -Rf^2 - c > -A_0$. Thus a negative V with $|V| > 1$ is impossible. The values $V = \pm 1$ are excluded by $V \equiv -c \pmod f$ and $f > 2c > 2$. Hence $V > 0$.

Define integers

$$o = cT - Rf, \qquad j = Tf - R\Delta i^2 c^3 - 1. \tag{22}$$

Then $of = V + c > 0$ and $jc = V + R > 0$, so $o, j > 0$ and

$$V = of - c = jc - R.$$

This restores the old auxiliary coordinates with their required positivity, before invoking the parent universal theorem.

5.4 The main Pell rank and the remaining signs

Write $A = a + 2$ and define $\chi_A(n), \psi_A(n)$ by

$$(A + \sqrt{A^2 - 1})^n = \chi_A(n) + \psi_A(n)\sqrt{A^2 - 1}.$$

Let $P_* = 2XY^2 + 1$. The positive first and main norms classify as

$$\tau = \chi_{P_*}(n), \quad k = 2\psi_{P_*}(n), \qquad D = \chi_A(p), \quad c = \psi_A(p),$$

with $n, p \geq 1$. Put $\epsilon = N_4$. Since $P_* \equiv 1 \pmod{E}$, $2n \equiv R + \epsilon \pmod{E}$. Equations (19) and (20) give $n \geq (R-1)/2 \geq 24$. As $P_* > A$ and $c > k$, Pell monotonicity gives $p > n$; consequently $c > A\Delta^2$, $c > 2p$, and $c > 2R$.

At this point we use two precisely identified local lemmas from the inherited proof, not its full input-decoding theorem. The strong-rank lemma of [7] applies with $i_{\mathrm{old}} = \Delta i$, because

$$(i_{\mathrm{old}} c^2)^2 = \Delta(f^2 - 1).$$

It gives $f = \chi_A(m)$, $c \mid m$, and $m \geq c > 2p$. The signed auxiliary step-down argument in [8, Section 4] then applies to $V = of - c = jc - R > 0$, using $0 < R < c/2$ and $0 < p < c/2$, and yields

$$p = R. \tag{23}$$

Neither local lemma uses the removed input norm. Their hypotheses and this application were independently reviewed for this report.

Write $2n = R + \epsilon + vE$ with $v \geq 0$. If $v \geq 1$, then $2n > 2R$, contradicting $n < p = R$. Thus $2n = R + \epsilon$. If $\epsilon = -1$, then $R = 2n + 1$. The duplication identity and $\chi_A(2) > P_*$ imply

$$\psi_A(2n) = 2A\psi_{\chi_A(2)}(n) \geq Ak.$$

Since $A > Y + 1$, we would have $c = \psi_A(2n+1) > k(Y+1)$, contrary to $c = kY + \eta$ with $0 < \eta < k$. Therefore $N_4 = 1$, and Lemma 5.1 gives $N_5 = 1$. All seven equalities in (13) have now been recovered.

5.5 Recovering a power without using the omitted input block

This step prevents a circular appeal to the old input theorem. Write $R = 2r + 1$, so $n = r + 1$ and $r \geq 1$. Elementary Pell growth gives

$$\frac{c}{k} \geq \frac{1}{2}\left(\frac{(2A-1)^2}{2P_*}\right)^r > \frac{1}{2}\left(\frac{(X+1)^2}{X}\right)^r.$$

The strict inequality follows by expansion using $a = Y(X+1)$. Together with $c/k < Y + 1$ and $X \geq 16$, it implies $Y > X^r/3$ and

$$a = Y(X+1) > X^{r+1}/3 > 2 \cdot 4^r = 2^R.$$

Set $E_A(j) = \chi_A(j) - a\psi_A(j)$. It starts with $1, 2$, obeys the Pell recurrence, and satisfies

$$E_A(j) \equiv 2^j \pmod{H}. \tag{24}$$

Indeed $H = 4A - 5$, so $2A\,2^{j+1} - 2^j \equiv 2^{j+2} \pmod{H}$. The recurrence therefore gives the same values modulo H. The retained definition of D says $E_A(R) = X + \gamma H$. Both X and 2^R lie strictly between 0 and $a < H$, so

$$X = 2^R. \tag{25}$$

Since $q \mid X$, write $q = 2^v$. The divisibility $2^d - 1 \mid 2^v - 1$ implies $d \mid v$, so $q = B^N$ for $N \geq 1$. Put $u_0 = b + 2d$. The fixed marker gives $2^{u_0} = W_0 < q = 2^v$; and (19) gives $v < R$. Thus $u_0 < R$. Both are odd, so $R \geq u_0 + 2$.

5.6 Restoring the three deleted input witnesses

Define

$$\kappa = \psi_A(u_0), \qquad \mu_0 = \chi_A(u_0),$$
$$\delta = \frac{\kappa - u_0}{\Delta}, \qquad \rho = \frac{\mu_0 - a\kappa - W_0}{H}, \qquad \sigma = \gamma - \rho. \tag{26}$$

The odd-index congruence $\psi_A(u_0) \equiv u_0 \pmod{\Delta}$ makes δ integral; $u_0 \geq 3$ and Pell growth make it positive. Congruence (24) makes ρ integral. Also $E_A(u_0) > \psi_A(u_0) > 2^{u_0}$, so $\rho > 0$.

To prove the remaining positivity, observe that

$$H\sigma = E_A(R) - E_A(u_0) - X + W_0.$$

Using $R \geq u_0 + 2$ and $\psi_A(j) < E_A(j) \leq 2\psi_A(j)$ for the relevant positive indices,

$$E_A(R) - E_A(u_0) > \psi_A(u_0 + 2) - 2\psi_A(u_0) > (4A^2 - 2A - 3)\psi_A(u_0) > A > X.$$

Hence $\sigma > 0$. The restored parent input quantities satisfy

$$\kappa = u_0 + \delta\Delta, \qquad \mu_0 = W_0 + a\kappa + \rho H, \qquad \mu_0^2 - \Delta\kappa^2 = 1.$$

All its other factors are identical, and $\rho + \sigma = \gamma$. Thus every positive zero of U gives a positive zero of the exact parent polynomial at $x = 1$. We may now apply that parent's complete ordinary-input acceptance theorem.

Conversely a positive parent zero at $x = 1$ has $W = W_0$ by its proved input-decoding theorem. Erase δ, ρ, σ, replace the latter two by $\gamma = \rho + \sigma$, and use the affine guard. Every retained factor is one, giving $U = 0$. The parent's input-rank theorem makes the restoration unique. This establishes the full positive-zero correspondence.

Finally $\Delta = (a + 1)(a + 3) > 0$ on the entire positive supplied domain. Hence $P = \Delta U$ has precisely the same positive zeros. Combining the inherited compiler, Lemma 3.1, and the K215 calibration proves Theorem 1.1.

6 Operation counts and exact degrees

6.1 The complete evaluator

The normalized parent evaluator uses $85 = 48\text{M} + 37\text{A}$ operations. Replacing $\rho + \sigma$ by the supplied γ removes one addition. Fixing $x = 1$ removes the multiplication $2d\,x$. Its eleven input-loader instructions are replaced by two instructions computing $3W + c_0$. The resulting unscaled circuit has $74 = 42\text{M} + 32\text{A}$ operations.

The exact coefficient sharing from [6] saves a further multiplication. The unscaled strong/auxiliary block computes $S = \Delta i^2 c^4$ and $A_0 = \Delta S$. Instead compute

$$v_0 = i(\Delta c^2), \qquad A_0 = v_0^2, \qquad \widetilde{N}_6 = \Delta f^2 - A_0 = \Delta N_6.$$

The quantity Δc^2 is already paid for in the main norm. Use $\widetilde{N}_6$ in the final product and subtract Δ instead of one. This is exactly $P = \Delta U$ over every commutative ring, not just on solutions. The complete circuit, including its finalizer, therefore has 73 gates.

For the system, remove the six product multiplications and final subtraction from the 74-gate unscaled evaluator, and remove the two-gate affine guard. This leaves $65 = 35\text{M} + 30\text{A}$ operations producing the seven comparison pairs. Computing the seven residuals, their squares, and their sum adds 20 operations, so the direct SOS evaluator has $85 = 42\text{M} + 43\text{A}$ operations. Equalities are counted separately in the comparison model; their arithmetic cost is fully charged in the SOS version.

6.2 Uniform exact degree

Assign degree one to each supplied unknown and degree zero to every fixed compiler numeral. Put

$$Q = \beta J, \quad k_0 = \eta + \zeta, \quad C_1 = Q - F - Z - \alpha, \quad W_1 = Q - F - 2Z - \alpha,$$

and $T_1^{\mathrm{tr}} = wC_1 - tQ$. The respective factor degrees of $U + 1$ are

$$22,\ 18,\ 1,\ 60,\ 7,\ 2,\ 34,$$

whose sum is 144. The main norm's apparent top terms cancel:

$$D^2 - \Delta c^2 = (X + \gamma H)^2 + 2ac(X + \gamma H) - Hc^2.$$

Its degree is 18, not the larger bound obtained by blindly propagating gate degrees.

Multiplying the leading forms gives

$$U_{144} = -24Q^{83}h\gamma i^4 k_0^{13} w^{11} s^{24} T^2 f^2 W_1 T_1^{\mathrm{tr}}. \tag{27}$$

Since $\Delta_{12} = w^2 s^2 Q^8$, the scaled leader is

$$P_{156} = -24Q^{91}h\gamma i^4 k_0^{13} w^{13} s^{26} T^2 f^2 W_1 T_1^{\mathrm{tr}}. \tag{28}$$

For example the coefficient of

$$J^{92}h\gamma i^4 \eta^{13} w^{13} s^{26} T^2 f^2 \alpha t$$

is $-24\beta^{92} \neq 0$. Thus 156 is the exact degree uniformly for every admissible fixed compiler, not a sample-based estimate. The analogous argument gives exact degree 144 for U.

The largest comparison residual degree is 60, from the auxiliary norm. Its square supplies the unique degree-120 part of the direct SOS. These claims are independently corroborated by full dense univariate coefficient executions over two finite fields; those computations support, and do not replace, the leading-form proof.

7 Unrestricted integers and a small-degree system

The positive domain in the principal theorem must not be silently changed to all integers. In particular $\Delta = 0$ creates off-domain zeros of $P = \Delta U$ which have no computational meaning. We give explicit domain conversions.

7.1 One equation in forty-four unrestricted integers

The coordinates τ, y occur only squared. At a solution of (13), $\tau \neq 0$ because $\tau^2 = 1 + L(L + k) > 1$, and $y \neq 0$ because otherwise $A_0 V^2 = 1$ with $A_0 > 1$. Consequently these two roots may range over all integers. The unit-sign proof remains valid for signed roots: use $|\tau|$ in the negative-Pell argument and retain the same square congruences. Thus every zero on this partially signed domain has nonzero roots, and absolute values restore a positive solution. The remaining fourteen coordinates must stay positive.

Every positive integer has a representation

$$v = 1 + a^2 + b^2 + c(c + 1), \qquad a, b, c \in \mathbb{Z}. \tag{29}$$

Indeed $4(v-1)+1$ is not of the excluded three-square form $4^r(8s+7)$. In a three-square representation it has exactly one odd square modulo four, giving (29). Conversely its right-hand side is always positive for integer a, b, c. Substitute (29) into the fourteen positive coordinates and keep τ, y signed. This defines a single polynomial in $14 \cdot 3 + 2 = 44$ unrestricted integer unknowns, with exactly the required solvability equivalence. Its degree is 312. Restricting all the b, c parameters to zero verifies noncancellation of the doubled leading degree. The literal substitution costs $14(3\mathrm{M} + 4\mathrm{A})$, giving $171 = 83\mathrm{M} + 88\mathrm{A}$ total operations.

7.2 Forty-six quadratic equations in eighty-three integers

Starting from the 65-operation comparison circuit, inline every affine gate. Three multiplication gates are affine as well: βJ, $4a$, and $q\widehat{M}_F$. There remain 32 genuinely nonlinear multiplication-register equations. Every remaining comparison is affine in the supplied coordinates and these product registers. Solve the seven comparisons using the pivots

$$\tau^2\text{-register},\ D^2\text{-register},\ f^2\text{-register}, \text{auxiliary-product register},\ hE\text{-register}, t(q-1)\text{-register},\ \alpha.$$

Each pivot has coefficient 1 or -1. Substitution therefore preserves integer solutions and introduces no divisibility condition. All product equations remain quadratic.

For each of the fourteen positive coordinates use the particularly simple positivity equation

$$4v - 3 = a_v^2 + b_v^2 + c_v^2. \tag{30}$$

It is solvable exactly when $v \geq 1$, by the same three-square theorem; α is replaced by its already solved affine expression. The two signed roots need no positivity equation. The total is $32 + 14 = 46$ quadratic equations. There are $(16 + 32 - 7) + 42 = 83$ unrestricted integer variables.

The archive lists all 46 residual polynomials explicitly in `data/quadratic83.txt`, and serializes every coefficient and exponent in `data/quadratic83.json`. Appendix A prints their defining equations. Let them be $Q_1, \ldots, Q_{46}$. Then the single equation

$$\boxed{Q_1^2 + \cdots + Q_{46}^2 = 0} \tag{31}$$

has exactly the same integer solvability condition. It has exact degree four: the fourth power of the first positivity parameter for J has coefficient one. This quartic is a useful alternative when low degree matters more than a short evaluator or few variables.

8 Why the equivalence is a theorem of ZFC

The proof chain is

$$\begin{aligned} \exists \mathbf{x}\ P(\mathbf{x}) = 0 &\iff \text{the fixed compiled input is accepted} \\ &\iff \mathrm{Halt}(\mathrm{K215}, \mathrm{blank}) \\ &\iff \exists n\ \mathrm{Proof}_{\mathsf{ZFC}}(n, \bot). \end{aligned}$$

Each step has both directions. The coefficient recipe consists of finite sets, bounded enumerations, integer sums and products, and primitive finite-table constructions. The normalization and tableau arguments are induction on finite computations. The Pell lemmas, integer-unit arguments, and three-square theorem are ordinary number theory available in ZFC. The inherited K215 calibration

has an ordinary ZFC-formalizable mathematical proof, separately supported by its external proof-assistant endpoint [1, 2]. Thus these are proofs ZFC can carry out; their composition does not assume the consistency or soundness of ZFC.

The delivered proof is not a string of ZFC proof-code instructions, and the new 73-gate composition has not been formalized in Lean or Rocq. An external Lean theorem is not, merely by being printed, a derivation encoded in ZFC's proof syntax. The present claim uses the ordinary mathematical arguments and their formalizability, and explicitly records which substantial inherited lemmas it uses.

Negating the existential equivalence gives the requested result. If ZFC is consistent, the equation has no standard positive solution, and ZFC cannot prove that lack of solutions, by its second incompleteness theorem. We do not infer full two-sided unprovability from consistency alone: ruling out a ZFC proof of the false existential assertion uses the relevant soundness assumption, such as Σ_1-soundness. The requested nonexistence–consistency equivalence does not require such an additional assumption.

9 Verification, reproducibility, and remaining work

The package has three distinct verification layers.

Literal machine and compiler data. The normalizer authenticates the exact K215 table, checks all 430 inherited transitions and the complete 660-row normalized table, checks several input-erasure lengths, and compares 120,000 paired simulation steps. These prefix checks test the implementation; they are not evidence of unconditional nonhalting. The compiler includes a two-sided 126-cell accepting tableau fixture, exact marker separation, both overlap directions, and five actually materialized small compiler instances. A separate source audit compares 25 sparse layout attributes against the two pinned ancestor compiler functions, including both optional-correction branches. None of these tests materializes a K215 accepting run. The factored window count was independently compared with all 209,952 weighted cross assignments for an 18-tile fixture; the scalar formulas agree with all five materialized compiler fixtures. The general counting derivation is given in the coefficient note.

Finite algebra. The supplied checker verifies complete source closure, liveness, interfaces, and operation ledgers. Eighty signed/positive arithmetic fixtures check the all-value identity $P = \Delta U$ and the exact SOS finalizer. Two full univariate coefficient evaluations check the degrees $144, 156, 120$. The quadratic compiler performs exact symbolic substitutions for all 32 product equations and all seven comparisons, verifies every variable is live, and verifies the coefficient-one quartic witness. These are exact finite checks, not a finite search used as a substitute for the unbounded mathematical equivalence.

Mathematics. A separate review within this research collaboration checked unit signs, pretyping bounds, auxiliary positivity, the exact strong-rank and step-down hypotheses, the noncircular recovery $X = 2^R$, and all three restored input witnesses. The final new theorem still relies on the explicitly cited inherited K215 and native compiler results. Fresh full builds of their Lean and Rocq dependency trees were not performed for this report.

From the archive directory, the standard-library checks run as follows:

```
python3 src/verify_all.py
```

The individual entry points are also available:

```
python3 src/diophantine.py --emit --verify
python3 src/normalize_k215_direct.py data/k215.tm --verify
python3 src/k215_constants.py data/normalized_k215.json --self-test
python3 src/quadratic_elimination.py --expect data/quadratic83.json
python3 src/audit_compiler_transcription.py
```

The TeX source compiles with two passes of pdfLaTeX. The archive also contains the generated JSON circuits, quadratic equations, verification receipts, source provenance, and a build script. The ordinary commands above are bounded practical checks; the opt-in full K215 coefficient expansion is excluded from them.

The next useful formalization step is to state a generic fixed-marker theorem for the native parent and prove the affine unit-guard reduction once. This avoids recompiling or re-proving the entire machine simulation for each arithmetic simplification. Coefficient height is a separate research objective: a compact direct register-machine or proof-calculus compiler could produce much smaller constants without improving the 73-gate count. The present package makes that boundary visible rather than hiding it in an unspecified universal index.

A The explicit quadratic equations

All symbols in the following listing are unrestricted integer unknowns except the six fixed coefficients `Bm1`, `Kconstant`, `twice_cell_bits`, `MC`, `MF`, and `W0`. They mean respectively $\beta, K_0, 2d, M_C, \widehat{M}_F, W_0$ from Section 4. Each expression is set equal to zero. The labels at the left identify eliminated source-register equations; a label is not an additional variable. A machine-readable complete variable list accompanies the equations in the archive.

```
01. tau_square: 1 + first_product - tau_root^2 = 0
02. Lbig: -1 - Bm1^2*Jrep^2 - 2*Bm1*Jrep + Lbig = 0
03. n2: -Bm1*Jrep*Lbig - Lbig + n2 = 0
04. wn2: -Bm1*Jrep*w - w + wn2 = 0
05. sn2: -n2*s + sn2 = 0
06. UM: UM - sn2*wn2 = 0
07. ksn2: -eta*sn2 + ksn2 - sn2*zeta = 0
08. first_root_base: -UM*ksn2 + first_root_base = 0
09. first_product: -eta*first_root_base + first_product - first_root_base^2 - first_root_base*zeta = 0
10. cam2: -UM*eta - UM*ksn2 + cam2 - eta*sn2 - ksn2*sn2 = 0
11. gam: -4*UM*gamma + gam - 3*gamma - 4*gamma*sn2 = 0
12. L15: 1 + Ac2 - cam2^2 - 2*cam2*gam - 2*cam2*wn2 - gam^2 - 2*gam*wn2 - wn2^2 = 0
13. a_square: -UM^2 - 2*UM*sn2 + a_square - sn2^2 = 0
14. c2: c2 - eta^2 - 2*eta*ksn2 - ksn2^2 = 0
15. Ac2: Ac2 - 4*UM*c2 - a_square*c2 - 3*c2 - 4*c2*sn2 = 0
16. ic2: -c2*i + ic2 = 0
17. ic22: -ic2^2 + ic22 = 0
18. strong_difference: -4*UM*ic22 - a_square*ic22 - 3*ic22 - 4*ic22*sn2 + strong_difference = 0
19. R16: R16 - 4*UM*strong_difference - a_square*strong_difference - 4*sn2*strong_difference - 3*
    strong_difference = 0
20. aux_y2: aux_y2 - y_aux^2 = 0
21. hpm1: -1 - UM*h + eta - mask - rproduct + zeta = 0
22. gap_product: -Bm1^2*Jrep^2 + Bm1*F*Jrep - Bm1*Jrep + gap_product = 0
23. rproduct: 1 + Bm1*Jrep - Bm1*Jrep*Lbig - F + F*Lbig - Lbig + Lbig*Z - Lbig*gap_product - Z + gap_product +
      rproduct = 0
24. mask: -Bm1*Jrep^2*MF - Jrep*MC - Jrep*MF + mask = 0
25. innerC: -Kconstant*W0 - Kconstant*Z - W0*w - Z*w + innerC = 0
26. local_rhs: Bm1*Jrep - Bm1*Jrep*transport_quotient - F + innerC = 0
27. L16: 1 - f^2 + strong_difference = 0
28. auxiliary_Tf: auxiliary_Tf - auxiliary_quotient*f = 0
29. auxiliary_c_Tf: -auxiliary_Tf*eta - auxiliary_Tf*ksn2 + auxiliary_c_Tf + eta + ksn2 = 0
30. auxiliary_R_f2: auxiliary_R_f2 - mask - mask*strong_difference - rproduct - rproduct*strong_difference = 0
```

```
31. H2: H2 - auxiliary_R_f2^2 + 2*auxiliary_R_f2*auxiliary_c_Tf - auxiliary_c_Tf^2 = 0
32. L17: 1 - H2*R16 + R16*aux_y2 - aux_y2 = 0
33. positive_Jrep: -3 + 4*Jrep - pos_Jrep_1^2 - pos_Jrep_2^2 - pos_Jrep_3^2 = 0
34. positive_F: -3 + 4*F - pos_F_1^2 - pos_F_2^2 - pos_F_3^2 = 0
35. positive_alpha: 1 + 4*Bm1*Jrep - 4*F - 4*W0 - 8*Z - pos_alpha_1^2 - pos_alpha_2^2 - pos_alpha_3^2 - 4*
    twice_cell_bits = 0
36. positive_transport_quotient: -3 - pos_transport_quotient_1^2 - pos_transport_quotient_2^2 -
    pos_transport_quotient_3^2 + 4*transport_quotient = 0
37. positive_f: -3 + 4*f - pos_f_1^2 - pos_f_2^2 - pos_f_3^2 = 0
38. positive_h: -3 + 4*h - pos_h_1^2 - pos_h_2^2 - pos_h_3^2 = 0
39. positive_i: -3 + 4*i - pos_i_1^2 - pos_i_2^2 - pos_i_3^2 = 0
40. positive_auxiliary_quotient: -3 + 4*auxiliary_quotient - pos_auxiliary_quotient_1^2 -
    pos_auxiliary_quotient_2^2 - pos_auxiliary_quotient_3^2 = 0
41. positive_s: -3 - pos_s_1^2 - pos_s_2^2 - pos_s_3^2 + 4*s = 0
42. positive_w: -3 - pos_w_1^2 - pos_w_2^2 - pos_w_3^2 + 4*w = 0
43. positive_eta: -3 + 4*eta - pos_eta_1^2 - pos_eta_2^2 - pos_eta_3^2 = 0
44. positive_zeta: -3 - pos_zeta_1^2 - pos_zeta_2^2 - pos_zeta_3^2 + 4*zeta = 0
45. positive_Z: -3 + 4*Z - pos_Z_1^2 - pos_Z_2^2 - pos_Z_3^2 = 0
46. positive_gamma: -3 + 4*gamma - pos_gamma_1^2 - pos_gamma_2^2 - pos_gamma_3^2 = 0
```

References

[1] ProveIt, *Formal correctness of the exact 215-state machine*, snapshot `cdd582bec13f908df3711fa9e05d1ddac368a5db`. Pinned K215 verification overview.

[2] ProveIt, *KEndpoint215.lean*, same snapshot. Literal Lean endpoint and dependency composition.

[3] ProveIt, *A 75-operation fixed-index universal certificate*, `FIXED_RAW_UNIVERSAL_75_PROOF.md`, same snapshot. Full native mathematical proof.

[4] ProveIt, *Modified half-binomial compiler*, `complete75_half_binomial_compiler.md` and its Python source, same snapshot. Compiler proof and fixed-numeral conventions.

[5] ProveIt, *An auxiliary quotient gives a complete universal polynomial in 85 operations*, `complete85_auxiliary_bezout_projection.md`, same snapshot. Normalized parent, positivity, and input interface.

[6] ProveIt, *A complete universal polynomial in 84 operations*, `complete84_scaled_strong_output.md`, same snapshot. Exact positive scaling and coefficient sharing.

[7] ProveIt, *Relaxed auxiliary Pell proof*, `PELL_RELAXED_AUXILIARY_PROOF.md`, same snapshot. Local strong-rank lemma.

[8] ProveIt, *Reversed auxiliary refinement*, `complete75_reversed_auxiliary89.md`, same snapshot. Signed auxiliary step-down argument.

[9] J. P. Jones, *Universal Diophantine equation*, Journal of Symbolic Logic **47** (1982), 549–571. doi:10.2307/2273588.

[10] J. Bayer and M. David, *A Formal Proof of Complexity Bounds on Diophantine Equations*, ITP 2025, LIPIcs 352, Article 3. doi:10.4230/LIPIcs.ITP.2025.3.